

NegotiatorX: A Robust Non-LLM Agent for HAN 2026

Serhat Giydiren

Department of Computer Science, Ozyegin University

`serhat.giydiren@ozu.edu.tr`

June 2026

Abstract

This report presents `NegotiatorX`, my final agent for the HAN 2026 league. HAN agents must negotiate with formal offers and also communicate with text messages. I chose a non-LLM design because the agent must be fast, reproducible, and stable under many different opponents. The final agent uses deterministic bidding, conservative acceptance, opponent behavior modeling, and short template-based messages. The main goal was not only to get high utility in easy cases, but also to avoid very low scores in difficult negotiations. The final version keeps one public class, `NegotiatorX`, and includes a late-stage guard for lower-tail failures. Local tests and leaderboard-inspired proxy tests were used to decide which changes were safe enough for the final submission.

Keywords: automated negotiation, human-agent negotiation, HAN 2026, opponent modeling, NegMAS, rule-based agent

1 Introduction

The selected competition is the Human-Agent Negotiation (HAN) 2026 league of ANAC. The task is to build an agent that can negotiate under the alternating offers protocol and also send human-readable messages [1, 2]. This is different from a normal automated negotiation task, because the agent should not only make good offers. It should also look consistent and understandable during the negotiation.

My final agent is named `NegotiatorX`. It is submitted as `negotiatorx.negotiator.NegotiatorX`. The agent does not use a large language model, deep learning, or reinforcement learning at run time. Instead, it uses game-theoretic ideas, search over the outcome space, rule-based decision logic, simple economic reasoning, and small psychology-inspired communication rules. This choice makes the behavior easier to test and less risky in hidden tournament settings.

2 Design Overview

The agent has five main parts. First, it builds an internal catalog of legal outcomes and sorts them by our own utility. Second, it updates an opponent model from the offers received during the negotiation. Third, it classifies the opponent’s behavior, for example whether the opponent is repeating similar offers, conceding, or acting unpredictably. Fourth, it chooses the next offer using both our utility and the estimated chance that the opponent may accept. Fifth, it decides whether to accept, reject, or continue with a safer counter offer.

The agent uses natural-language messages, but they are generated from templates. This avoids slow or unstable LLM calls. The messages are short and connected to the formal offer. I also changed the messages so they do not show raw Python tuples to the user. When possible, the message refers to issue names and a clear reason for the move.

3 Negotiation Strategy

The bidding strategy is based on phases. In the early stage, the agent starts from strong offers. In the middle stage, it searches for bridge offers that keep our utility acceptable while becoming more realistic for the opponent. In the late stage, it becomes more focused on agreement, but it still protects against very weak deals.

The opponent model is a recency-weighted frequency model. If the opponent keeps asking for the same value on an issue, the agent treats that value as more important for the opponent. The model is not assumed to be perfect. It is used as a signal, not as the truth. This is important because some opponents concede because of time pressure, not because their real preference changed.

The acceptance policy is conservative. The agent accepts an offer when it is at least as good as the expected value of continuing, or when the negotiation is late and the offer is a safe agreement above the protected floor. The final version also includes a lower-tail guard inspired by our ANL experiments. This guard is only active very late and only when the negotiation looks stalled. Its purpose is to reduce cases where the agent gets a very low score because no good agreement is reached.

The design follows the general idea of time-dependent negotiation decision functions [4], but it adds behavior-based safeguards for the HAN setting. It also uses opponent modeling ideas similar to the surveyed approaches in automated bilateral negotiation [3].

4 Implementation

The implementation is kept inside one public agent class. This was important because the competition loader requires a standard Python module path and class name. The final identity is:

Agent module	<code>negotiatorx.negotiator</code>
Class name	<code>NegotiatorX</code>
Runtime LLM use	No
Main technologies	Game theory, search, rules, optimization, psychology

During development I tested several variants, including stricter acceptance, different issue-weight estimates, uncertainty smoothing, contextual profile selection, and more aggressive late rescue behavior. I did not promote every idea to the final agent. Some variants improved one local test but made other tests worse. For the final version, I kept the changes that improved robustness without changing the public agent identity.

5 Evaluation

The agent was tested with the official HAN scenarios, the provided baseline opponents, generated scenarios, and proxy opponents inspired by public leaderboard results. The main lesson was that average score alone was not enough. Some strategies looked good on average but failed badly in a few negotiations. Because of this, I also looked at lower-tail behavior, zero-score risk, agreement rate, median score, and first-quartile score.

The final promoted version kept the official local tournament score at 0.627292. It also improved several internal lower-tail gates compared with the previous default profile. These tests were useful because public leaderboard rankings changed a lot between runs. A strategy that is only tuned for one small local set can easily become unstable when the opponent pool changes.

6 Conclusion

`NegotiatorX` is a deterministic, non-LLM negotiation agent for HAN 2026. The main idea is to combine strong self-utility protection with opponent behavior modeling and late-stage robustness. The final version does not try to win by one fixed trick. It tries to stay competitive across many opponent types and avoid very weak outcomes in hard cases. If I had more time, I would focus on human evaluation of the messages and a larger hidden-opponent proxy set, because those two parts are still the hardest to measure locally.

References

- [1] ANAC 2026 Human-Agent Negotiation Call for Participation. <https://anac.cs.brown.edu/files/han/y2026/2026cfp.pdf>, 2026. Accessed during the project.
- [2] HAN 2026 Tutorial and Agent Template. <https://anac.cs.brown.edu/files/han/y2026/template2026.pdf>, 2026. Accessed during the project.
- [3] Tim Baarslag, Mark J. C. Hendriks, Koen V. Hindriks, and Catholijn M. Jonker. Learning about the opponent in automated bilateral negotiation: A comprehensive survey of opponent modeling techniques. *Autonomous Agents and Multi-Agent Systems*, 30:849–898, 2016.
- [4] Peyman Faratin, Carles Sierra, and Nicholas R. Jennings. Negotiation decision functions for autonomous agents. *Robotics and Autonomous Systems*, 24(3):159–182, 1998.